

Contenido

Introducción:	2
Previo:	2
Ejecución:	3
Paso 1. Introducimos este código en las celdas correspondientes:	3
Paso 2. Introducimos código para leer fichero (importante cambiar la ruta de tu bucket de s3 marcada en rojo). Una vez leído , lo mostramos	4
Paso 3 Imprimimos esquema de Dataframe	5
Paso 4 Seleccionamos columnas del Dataframe con el comando select	7
Paso 5 Seleccionamos columnas del Dataframe con el comando select y manipulamos los datos de una de ellas incrementando con 1 su valor	7
Paso 6 Filtramos columnas del Dataframe por valor con el comando filter	8
Paso 7. Hacemos la primera agregación del dataframe , contando cuantos valores hay de cada VendorID con el comando count	10
Paso 8: Hacemos el mismo conteo del dataframe , contando cuantos valores hay pero esta vez por el campo type_trip	10
Paso 9 Queremos hacer una suma , pero esta debe ser por un campo numérico , así que convertimos el campo passenegr_count a integer y trip_distance a double	10
Paso 10 Hacemos esa suma por el campo Passenger_count con la función sum	10

Reto 4. EJEMPLO SPARK CON EMR EJERCICIO DATAFRAME

Introducción:

Vamos a ver, a continuación, un ejemplo completo resuelto en el que usaremos Data Frame

Para ello, vamos a usar los ficheros 'tripdata.csv' , almacenados ambos en un bucket de S3

El objetivo es aprender las principales instrucciones de pyspark para manejo y uso de Dataframes

Previo:

Antes de empezar lo primero es arrancar pyspark, para ello crearemos un cluster en EMR con spark, jupyter , jupytergateway como aplicaciones seleccionadas para su instalación y podremos ejecutar este ejercicio de RDD desde un JupyterNotebook., como se muestra en la siguiente imagen

Servicios

Cluster EMR UAX

Versión de Amazon EMR

Información

Una versión contiene un conjunto de aplicaciones que se puede instalar en el clúster.

emr-7.1.0

Paquete de aplicaciones

Spark Interactive



Core Hadoop



Flink



HBase



Presto



Trino



Custom



☐ AmazonCloudWatchAgent 1.300032.2
 ☐ Flink 1.18.1
 ☐ HBase 2.4.17

☐ HCatalog 3.1.3
 ☐ Hadoop 3.3.6
 ☐ Hive 3.1.3

☐ Hue 4.11.0
 ☒ JupyterEnterpriseGateway 2.6.0
 ☒ JupyterHub 1.5.0

☒ Livy 0.8.0
 ☐ MXNet 1.9.1
 ☐ Oozie 5.2.1

☐ Phoenix 5.1.3
 ☐ Pig 0.17.0
 ☐ Presto 0.284

☒ Spark 3.5.0
 ☐ Sqoop 1.4.7
 ☐ TensorFlow 2.11.0

☐ Tez 0.10.2
 ☐ Trino 435
 ☐ Zeppelin 0.10.1

☐ ZooKeeper 3.9.1

Configuración del Catálogo de datos de AWS Glue

Utilice el Catálogo de datos de AWS Glue para proporcionar un meta-almacén externo a la aplicación.

☒ Usar para metadatos de la tabla de Spark

Opciones del sistema operativo

Información

☒ Versión de Amazon Linux
 ☐ Imagen de máquina de Amazon (AMI) personalizada

☒ Aplicar automáticamente las actualizaciones más recientes de Amazon Linux

IMPORTANTE tener en cuenta lo siguiente:

- Máquinas m4.large
- Roles de EMR

Recordar que una vez activo el cluster EMR , hay que crear el studio y workspace desde el menú de la izquierda y asociar este workspace al cluster de EMR creado

Ejecución:

Paso 1. Introducimos este código en las celdas correspondientes:

Crear la sesión de Spark Sparksession

El punto de entrada a todas las funciones de Spark es la clase SparkSession. Para crear una SparkSession básica, simplemente use SparkSession.builder:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
```

Paso 2. Introducimos código para leer fichero (importante cambiar la ruta de tu bucket de s3 marcada en rojo). Una vez leído , lo mostramos

Con SparkSession, las aplicaciones pueden crear DataFrames a partir de un RDD existente, de una tabla de Hive o de fuentes de datos de Spark.

Como ejemplo, lo siguiente crea un DataFrame basado en el contenido del archivo csv de tripdata que hemos subido a un bucket (importante que cambies la instrucción para poner tu ruta):

```
# spark is an existing SparkSession

df = spark.read.csv("s3://yourpath/tripdata.csv")

# Displays the content of the DataFrame to stdout

df.show()
```

VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	store_and_fwd_flag	RatecodeID	PULocationID	DOLocationID	passenger_count	trip_distance	fare_amount	extra_mta_tax	tip_amount	tolls_amount	ehail_fee	improvement_surcharge	total_amount	payment_type	trip_type
1	2	1/1/17 0:01		1	1	1/1/17 0:11	1	1.71	9	0	0.5	0	0.3	0.3	9.8	0	
1	2	1/1/17 0:03		1	1	1/1/17 0:09	1	1.44	6.5	0.5	0.5	0	0.3	0.3	7.8	0	
1	2	1/1/17 0:04		1	1	1/1/17 0:12	5	3.45	12	0.5	0.5	0	0.3	0.3	15.96	0	
1	2	1/1/17 0:01		1	1	1/1/17 0:14	1	2.11	10.5	0.5	0.5	0	0.3	0.3	11.8	0	
1	2	1/1/17 0:00		1	1	1/1/17 0:18	1	2.76	11.5	0.5	0.5	0	0.3	0.3	12.8	0	
1	2	1/1/17 0:00		1	1	1/1/17 0:13	1	4.14	15	0.5	0.5	0	0.3	0.3	15.8	0	

Aquí incluimos algunos ejemplos básicos de procesamiento de datos estructurados utilizando dataframes y Python.

En Python es posible acceder a las columnas de un DataFrame ya sea por atributo (df.total_amount,) o por indexación (df['total_amount,']). Si bien el primero es conveniente para la exploración interactiva de datos, se recomienda encarecidamente a los alumnos que utilicen el segundo formulario, que está preparado para el futuro y no romperá con los nombres de columnas que también son atributos en la clase DataFrame.

En la siguiente instrucción, imprimimos esquema de DataFrames

```
# spark, df are from the previous example
# Print the schema in a tree format
df.printSchema()

root

|-- vendorID: string (nullable = true)
|-- lpep_pickup_datetime: string (nullable = true)
|-- lpep_dropoff_datetime: string (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- RatecodeID: string (nullable = true)
|-- PULocationID: string (nullable = true)
|-- DOLocationID: string (nullable = true)
|-- passenger_count: string (nullable = true)
|-- trip_distance: string (nullable = true)
|-- fare_amount: string (nullable = true)
|-- extra: string (nullable = true)
|-- mta_tax: string (nullable = true)
|-- tip_amount: string (nullable = true)
|-- tolls_amount: string (nullable = true)
|-- ehail_fee: string (nullable = true)
|-- improvement_surcharge: string (nullable = true)
|-- total_amount: string (nullable = true)
|-- payment_type: string (nullable = true)
```

```
-- trip_type: string (nullable = true)
```

Paso 4 Seleccionamos columnas del Dataframe con el comando select

```
# Select only the "VendorID" column
```

```
df.select(df['VendorID']).show()
```

```
+-----+  
|VendorID|
```

```
+-----+
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
|      2|
```

```
+-----+
```

```
only showing top 20 rows
```

Paso 5 Seleccionamos columnas del Dataframe con el comando select y manipulamos los datos de una de ellas incrementando con 1 su valor

```
# Select Vendor_id, but increment the total_amount by 1
```

```
df.select(df['VendorID'], df['total_amount'] + 1).show()
```

```
Spark Job Progress
```

```
+-----+-----+
```

```
|VendorID|(total_amount + 1)|
```

```
+-----+-----+
```

```
|      2|          10.8|
```

```
|      2|           8.8|
```

```
|      2|         16.96|
```

```
|      2|          12.8|
```

```
|      2|          13.8|
```

```
|      2|          17.3|
```

	/	2/	21.3/
/	2/	13.3/	
/	2/	7.3/	
/	2/	15.8/	
/	2/	12.76/	
/	2/	24.3/	
/	2/	32.3/	
/	2/	9.3/	
/	2/	26.74/	
/	2/	12.3/	
/	2/	4.0/	
/	2/	13.36/	
/	2/	22.59/	
/	2/	12.8/	

Paso 6 Filtramos columnas del Dataframe por valor con el comando filter

```
# Select people older than 21
```

```
df.filter(df['total_amount'] > 100).show()
```

```
-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+
```

```
/vendorID/lpep_pickup_datetime/lpep_dropoff_datetime/store_and_fwd_flag/RatecodeID/PULocationID/DOLocationID/passenger_count/trip_distance/fare_amount/extra_mta_tax/tip_amount/tolls_amount/ehail_fee/improvement_surcharge/total_amount/payment_type/trip_type/
```

```
+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+-----+
```

```
/      2/      1/1/17 1:31/      1/1/17 2:14/      N/
5/      181/      265/      3/      13.66/      105/      0/
0/      0/      0/      NULL/      0/      105/
1/      2/
```



```

/      2/      1/1/17 1:12/      1/1/17 2:47/
N/      4/      223/      265/      1/      24.11/      119
.5/  0.5/  0.5/      0/      10.5/      NULL/      0.3/
131.3/      1/      1/

/      2/      1/1/17 1:27/      1/1/17 3:14/      N/
1/      260/      161/      1/      29.82/      96/  0.5/
0.5/      25.71/      5.54/      NULL/      0.3/      128.55/
1/      1/

/      2/      1/1/17 2:26/      1/1/17 3:18/      N/
4/      16/      265/      1/      19.73/      90/  0.5/
0.5/      18.26/      0/      NULL/      0.3/      109.56/
1/      1/

/      2/      1/1/17 2:32/      1/1/17 3:11/      N/
5/      257/      265/      4/      22.01/      120/  0/
0/      0/      21.58/      NULL/      0/      141.58/
1/      2/

/      2/      1/1/17 4:26/      1/1/17 4:26/      N/
5/      223/      223/      1/      0/      200/  0/
0/      40/      0/      NULL/      0/      240/
1/      2/

/      2/      1/1/17 5:37/      1/1/17 6:10/      N/
4/      255/      1/      5/      18.72/      84/  0.5/
0.5/      19.16/      10.5/      NULL/      0.3/      114.96/
1/      1/

/      2/      1/1/17 5:57/      1/1/17 7:47/      N/
1/      108/      178/      1/      30.12/      106/  0.5/
0.5/      0/      10.5/      NULL/      0.3/      117.8/
2/      1/

/      2/      1/1/17 5:34/      1/1/17 6:07/      N/
4/      130/      265/      3/      30.27/      133/  0.5/
0.5/      0/      0/      NULL/      0.3/      134.3/
1/      1/

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+

# Count people by age

df.groupBy("VendorID").count().show()

+-----+-----+
/VendorID/count/
+-----+-----+
/      1/ 3687/
/      2/16313/

```

--	--

Paso 7. Hacemos la primera agregación del dataframe , contando cuantos valores hay de cada VendorID con el comando count

```
df.groupBy("VendorID").count().show()
```

Spark Job Progress

```
+-----+-----+
|VendorID|count|
+-----+-----+
|      1| 3687|
|      2|16313|
+-----+-----+
```

Paso 8: Hacemos el mismo conteo del dataframe , contando cuantos valores hay pero esta vez por el campo type_trip

```
df.groupBy("trip_type").count().show()
```

```
+-----+-----+
|trip_type|count|
+-----+-----+
|      1|19632|
|      2|  368|
+-----+-----+
```

Paso 9 Queremos hacer una suma , pero esta debe ser por un campo numérico , así que convertimos el campo passenger_count a integer y trip_distance a double

```
from pyspark.sql.types import *
from pyspark.sql.functions import col
from pyspark.sql.types import StringType, BooleanType, DateType
df2 = df.withColumn("VendorID", col("VendorID").cast(StringType())) \
        .withColumn("passenger_count", col("passenger_count").cast(IntegerType())) \
        .withColumn("trip_distance", col("trip_distance").cast(DoubleType()))
df2.printSchema()
```

Paso 10 Hacemos esa suma por el campo Passenger_count con la función sum

```
from pyspark.sql.functions import col
from pyspark.sql.types import StringType, BooleanType, DateType
df2.groupBy("VendorID").sum("passenger_count").show()
```

Spark Job Progress

```
+-----+-----+
|VendorID|sum(passenger_count)|
+-----+-----+
```

		+-----+-----+
	1	4955
	2	23529
+-----+	+-----+	